

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to

solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

Duration : 45 Minutes

QUESTIONS: 2 Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

I _____HARSHA-192524106_____ (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:HARSHA

Annexure A

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify

NAME:S.Harshavardhan

Reg:192524106

Course:CSA1718-AI

SMART HOME AUTOMATION SYSTEM USING INTELLIGENT AGENTS

1. AIM

The aim of this program is to design and demonstrate a Smart Home Automation System using Intelligent Agent concepts to automatically control lighting, temperature, and security based on real-time environmental conditions and user preferences.

The system demonstrates the behavior of:

- Simple Reflex Agent
- Model-Based Agent
- Goal-Based Agent
- Utility-Based Agent

The main objectives are to improve user comfort, energy efficiency, and home security by making intelligent decisions based on sensor inputs such as motion, temperature, time of day, and security status.

2. PROBLEM STATEMENT

A smart home automation system must manage lighting, temperature, and security automatically based on user behavior and environmental conditions.

The system receives real-time inputs such as:

- Motion detection
- Current room temperature
- Time of day
- Door/security status
- User temperature preference

Different types of intelligent agents can be used to process this information.

Simple Reflex Agent

A Simple Reflex Agent reacts directly to the current input using condition-action rules.

Example:

IF motion is detected AND it is night → Turn ON light

It does not remember previous conditions.

Model-Based Agent

A Model-Based Agent maintains information about the current and previous state of the home.

For example, it can remember whether a room is occupied even when no movement is temporarily detected.

Goal-Based Agent

A Goal-Based Agent performs actions to achieve specific objectives.

For example:

Goal → Maintain comfortable room temperature

If the desired temperature is 24°C and the current temperature is 30°C, the system can turn on the AC to move toward the desired state.

Utility-Based Agent

A Utility-Based Agent compares different possible actions and selects the action that provides the best balance between:

- Comfort
- Energy efficiency
- Security

For example, instead of setting the AC to 18°C for maximum cooling, it may select 24°C because it provides good comfort while consuming less energy.

Recommended Approach

A Hybrid Intelligent Agent is most appropriate for the smart home.

It can combine:

Simple Reflex + Model-Based + Goal-Based + Utility-Based behavior

This allows the system to respond quickly to emergencies, remember environmental conditions, achieve specific goals, and optimize comfort and energy usage.

3. PYTHON CODE

```
# Smart Home Automation System
```

```
# Demonstration of Intelligent Agents
```

```
class SmartHomeAgent:

    def __init__(self):

        # Model-Based Agent memory
        self.room_occupied = False

        # User preference
        self.preferred_temperature = 24

        # -----
        # SIMPLE REFLEX AGENT
        # -----

    def simple_reflex(self, motion, time):

        if motion and time == "Night":
            return "Light ON"

        elif not motion:
            return "Light OFF"

        else:
            return "No Lighting Change"
```

```
# -----
```

```
# MODEL-BASED AGENT
```

```
# -----
```

```
def model_based(self, motion):
```

```
    # Update internal model
```

```
    if motion:
```

```
        self.room_occupied = True
```

```
    if self.room_occupied:
```

```
        return "Room Occupied"
```

```
    return "Room Empty"
```

```
# -----
```

```
# GOAL-BASED AGENT
```

```
# -----
```

```
def goal_based(self, temperature):
```

```
    goal = self.preferred_temperature
```

```
    if temperature > goal:
```

```
        return "Turn ON AC"
```

```
    elif temperature < goal:
```

```
        return "Turn ON Heater"
```

```
else:  
    return "Temperature Comfortable"
```

```
# -----  
# UTILITY-BASED AGENT  
# -----
```

```
def utility_based(self, temperature):
```

```
    difference = abs(  
        temperature -  
        self.preferred_temperature  
    )
```

```
    if difference >= 5:  
        return (  
            "AC at 24 C - "  
            "High Comfort and "  
            "Balanced Energy Usage"  
        )
```

```
    elif difference >= 2:  
        return (  
            "Use Fan - "  
            "Moderate Comfort and "  
            "Low Energy Usage"  
        )
```

```
else:
    return (
        "No Cooling Required - "
        "Save Energy"
    )
```

```
# -----
# SECURITY AGENT
# -----
```

```
def security(self, unauthorized_entry):

    # Immediate reflex action
    if unauthorized_entry:
        return (
            "ALERT! Unauthorized Entry - "
            "Security Alarm ON"
        )
```

```
    return "Home Secure"
```

```
# -----
# MAIN PROGRAM
# -----
```

```
home = SmartHomeAgent()
```

```
print("=" * 50)
print("SMART HOME AUTOMATION SYSTEM")
print("=" * 50)
```

```
# Sensor inputs
```

```
motion = True
```

```
time = "Night"
```

```
temperature = 29
```

```
unauthorized_entry = False
```

```
print("\nSENSOR INPUTS")
```

```
print("-----")
```

```
print("Motion Detected:", motion)
```

```
print("Time:", time)
```

```
print("Temperature:", temperature, "C")
```

```
print(
```

```
    "Unauthorized Entry:",
```

```
    unauthorized_entry
```

```
)
```

```
# Simple Reflex Agent
```

```
print("\n1. SIMPLE REFLEX AGENT")
```

```
print("-----")
```

```
lighting = home.simple_reflex(
```

```
    motion,
```

```

        time
    )

    print("Decision:", lighting)

# Model-Based Agent
print("\n2. MODEL-BASED AGENT")
print("-----")

occupancy = home.model_based(
    motion
)

print("Internal State:", occupancy)

# Goal-Based Agent
print("\n3. GOAL-BASED AGENT")
print("-----")

temperature_action = home.goal_based(
    temperature
)

print(
    "Goal Temperature:",
    home.preferred_temperature,
    "C"

```

```
)
```

```
print(  
    "Decision:",  
    temperature_action  
)
```

```
# Utility-Based Agent  
print("\n4. UTILITY-BASED AGENT")  
print("-----")
```

```
utility_action = home.utility_based(  
    temperature  
)
```

```
print(  
    "Best Utility Decision:",  
    utility_action  
)
```

```
# Security  
print("\n5. SECURITY SYSTEM")  
print("-----")
```

```
security_status = home.security(  
    unauthorized_entry  
)
```

```
print("Decision:", security_status)
```

```
print("\n" + "=" * 50)
```

```
print("FINAL SMART HOME STATUS")
```

```
print("=" * 50)
```

```
print("Lighting:", lighting)
```

```
print("Occupancy:", occupancy)
```

```
print("Temperature Control:",
```

```
    temperature_action)
```

```
print("Utility Decision:",
```

```
    utility_action)
```

```
print("Security:", security_status)
```

4. OUTPUT

When the above Python program is executed with the given sensor inputs, the following output is obtained:

```
=====
```

```
SMART HOME AUTOMATION SYSTEM
```

```
=====
```

```
SENSOR INPUTS
```

```
-----
```

```
Motion Detected: True
```

```
Time: Night
```

```
Temperature: 29 C
```

```
Unauthorized Entry: False
```

1. SIMPLE REFLEX AGENT

Decision: Light ON

2. MODEL-BASED AGENT

Internal State: Room Occupied

3. GOAL-BASED AGENT

Goal Temperature: 24 C

Decision: Turn ON AC

4. UTILITY-BASED AGENT

Best Utility Decision:

AC at 24 C - High Comfort and Balanced Energy Usage

5. SECURITY SYSTEM

Decision: Home Secure

=====

FINAL SMART HOME STATUS

=====

Lighting: Light ON

Occupancy: Room Occupied

Temperature Control: Turn ON AC

Utility Decision: AC at 24 C - High Comfort and

Balanced Energy Usage

Security: Home Secure

Output Explanation

The input conditions are:

Motion = True

Time = Night

Temperature = 29°C

Preferred Temperature = 24°C

Unauthorized Entry = False

Based on these conditions:

1. The Simple Reflex Agent detects motion during the night and switches the light ON.
2. The Model-Based Agent updates its internal state and records that the room is occupied.
3. The Goal-Based Agent compares the current temperature of 29°C with the preferred temperature of 24°C and decides to turn ON the AC.
4. The Utility-Based Agent selects AC at 24°C because it provides a balance between user comfort and energy efficiency.
5. The Security Agent determines that no unauthorized entry has occurred and reports that the home is secure.

Thus, the program demonstrates how different intelligent-agent concepts can work together in a smart home automation environment.

5. CONCLUSION

The Smart Home Automation System was successfully designed using different types of Intelligent Agents.

The Simple Reflex Agent provides immediate responses to current sensor inputs, such as switching the lights ON when motion is detected at night.

The Model-Based Agent maintains an internal state of the home, such as whether a room is occupied.

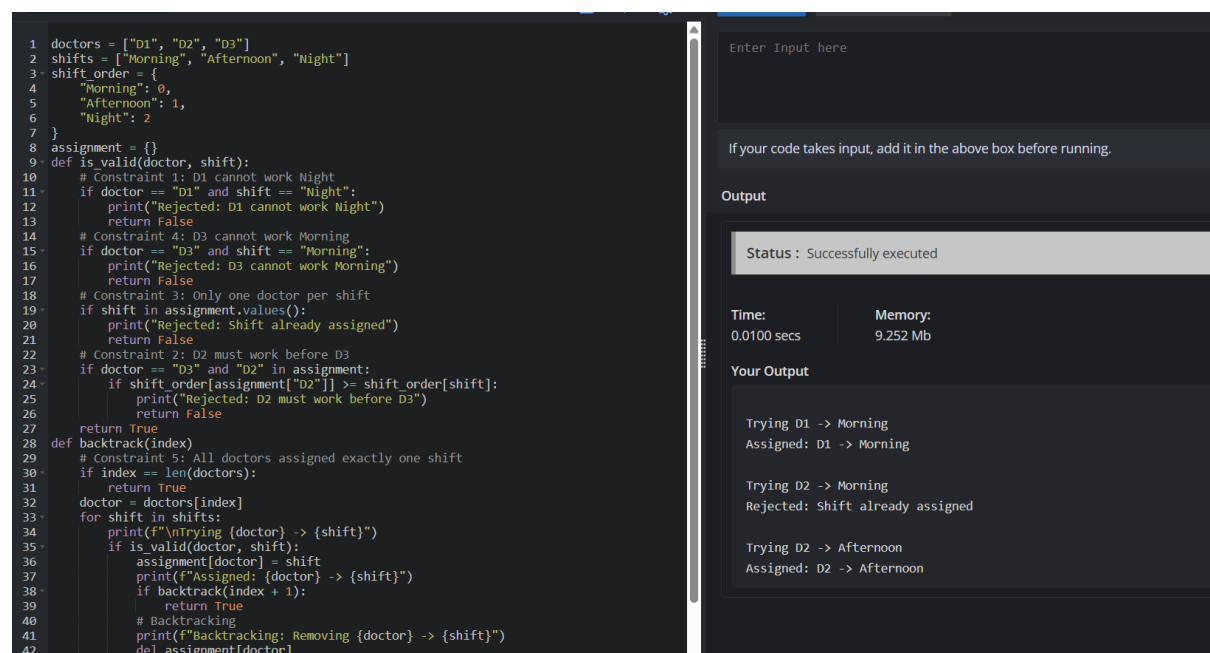
The Goal-Based Agent makes decisions to achieve a desired objective, such as maintaining the preferred temperature of 24°C.

The Utility-Based Agent evaluates different alternatives and selects an action that provides a suitable balance between comfort and energy efficiency.

For a practical smart home automation system, a hybrid intelligent-agent approach is the most suitable because it combines the advantages of all these agent types.

Therefore, the proposed hybrid system can effectively provide better comfort, improved energy efficiency, automatic environmental control, and enhanced home security.

Output:



```
1 doctors = ["D1", "D2", "D3"]
2 shifts = ["Morning", "Afternoon", "Night"]
3 shift_order = {
4     "Morning": 0,
5     "Afternoon": 1,
6     "Night": 2
7 }
8 assignment = {}
9 def is_valid(doctor, shift):
10     # Constraint 1: D1 cannot work Night
11     if doctor == "D1" and shift == "Night":
12         print("Rejected: D1 cannot work Night")
13         return False
14     # Constraint 4: D3 cannot work Morning
15     if doctor == "D3" and shift == "Morning":
16         print("Rejected: D3 cannot work Morning")
17         return False
18     # Constraint 3: Only one doctor per shift
19     if shift in assignment.values():
20         print("Rejected: Shift already assigned")
21         return False
22     # Constraint 2: D2 must work before D3
23     if doctor == "D3" and "D2" in assignment:
24         if shift_order[assignment["D2"]] >= shift_order[shift]:
25             print("Rejected: D2 must work before D3")
26             return False
27     return True
28 def backtrack(index):
29     # Constraint 5: All doctors assigned exactly one shift
30     if index == len(doctors):
31         return True
32     doctor = doctors[index]
33     for shift in shifts:
34         print(f"Trying {doctor} -> {shift}")
35         if is_valid(doctor, shift):
36             assignment[doctor] = shift
37             print(f"Assigned: {doctor} -> {shift}")
38             if backtrack(index + 1):
39                 return True
40             # Backtracking
41             print(f"Backtracking: Removing {doctor} -> {shift}")
42             del assignment[doctor]
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.252 Mb

Your Output

```
Trying D1 -> Morning
Assigned: D1 -> Morning

Trying D2 -> Morning
Rejected: Shift already assigned

Trying D2 -> Afternoon
Assigned: D2 -> Afternoon
```

ROBOT NAVIGATION IN AN UNKNOWN MAZE USING UCS AND IDS

1. AIM

The aim of this program is to design an intelligent robot capable of finding a path from a starting position to the exit of an unknown maze using uninformed search strategies.

The program demonstrates and compares:

- Uniform Cost Search (UCS)
- Iterative Deepening Search (IDS)

The system also demonstrates how a model-based, goal-oriented intelligent agent can maintain information about the maze and use search algorithms to reach the exit.

The major objectives are:

- Find a valid path from the start to the exit.
- Avoid obstacles and blocked cells.

- Demonstrate Uniform Cost Search.
 - Demonstrate Iterative Deepening Search.
 - Compare the paths generated by UCS and IDS.
 - Analyze time and space requirements.
 - Show how intelligent-agent design supports maze navigation.
-

2. PROBLEM STATEMENT

A robot is placed inside an unknown maze and must find a path to the exit without having prior knowledge of the complete maze layout.

The robot can encounter:

- Open paths
- Walls
- Dead ends
- Multiple possible routes
- Previously unexplored locations

Since the robot does not initially have heuristic information indicating the direction of the exit, uninformed search algorithms can be used.

Two appropriate uninformed search strategies are Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

Consider the following example maze:

S . X . .

. . X . .

X . . . X

. X X . .

. . . . G

Where:

S = Starting position

G = Goal/Exit

X = Wall or obstacle

. = Free path

The robot can move only:

Up, Down, Left, Right

Uniform Cost Search

Uniform Cost Search selects the path having the lowest accumulated cost.

It uses a priority queue.

If:

$g(n)$ = accumulated cost from S to node n

then UCS always expands the node having the smallest value of $g(n)$.

When every movement costs 1, UCS finds a shortest path in terms of number of movements.

Advantages of UCS

- Complete for positive step costs.
- Finds the minimum-cost solution.
- Supports different movement costs.
- Useful for cost-sensitive robot navigation.

Disadvantages of UCS

- Can consume significant memory.
- May store many nodes in its priority queue.
- Can be slow for very large search spaces.
- Does not use heuristic information about the goal.

Iterative Deepening Search

Iterative Deepening Search repeatedly performs Depth-Limited Search.

The depth limit is gradually increased:

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$

until the goal is found.

For example:

Depth 0:

S

Depth 1:

$S \rightarrow$ neighboring cells

Depth 2:

$S \rightarrow$ neighbors $\rightarrow$ next-level nodes

Depth 3:

Continue deeper...

IDS combines the low-memory advantage of DFS with the completeness of breadth-wise exploration.

Advantages of IDS

- Requires relatively little memory.
- Complete for finite branching factor.
- Useful when the goal depth is unknown.
- Finds a shallowest solution when all step costs are equal.

Disadvantages of IDS

- Repeatedly explores nodes near the start.
- Can take additional execution time because of repeated depth-limited searches.
- Does not directly optimize paths when movement costs are different.

Complexity Comparison

Let:

b = Branching factor

d = Depth of the shallowest solution

For IDS:

Time Complexity = $O(b^d)$

Space Complexity = $O(bd)$

For UCS, let:

C^* = optimal solution cost

ϵ = minimum positive step cost

A common worst-case bound is approximately:

Time = $O(b^{(1 + \text{floor}(C^*/\epsilon))})$

Space = $O(b^{(1 + \text{floor}(C^*/\epsilon))})$

Therefore, UCS may require considerably more memory, while IDS is attractive when memory is limited.

Intelligent Agent Design

The maze robot can be designed as a Model-Based Goal-Oriented Agent.

The model-based component stores information such as:

- Visited cells
- Discovered walls
- Dead ends
- Current robot location
- Available paths

The goal-based component has the objective:

Reach the Exit G

Therefore:

Sensors → Update Internal Maze → Search → Select Movement → Move → Sense Again

This allows the robot to gradually build knowledge of an initially unknown maze.

3. PYTHON CODE

```
import heapq
```

```
# -----
```

```
# UNKNOWN MAZE - UCS AND IDS
```

```
# -----
```

```
# S = Start
# G = Goal
# X = Obstacle
# . = Free Cell
```

```
maze = [
    ['S', '.', 'X', '.', '.'],
    ['.', '.', 'X', '.', '.'],
    ['X', '.', '.', '.', 'X'],
    ['.', 'X', 'X', '.', '.'],
    ['.', '.', '.', '.', 'G']
]
```

```
rows = len(maze)
cols = len(maze[0])
```

```
start = (0, 0)
goal = (4, 4)
```

```
# Up, Down, Left, Right
```

```
directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]
```

```
# -----
```

```
# GET VALID NEIGHBORS
```

```
# -----
```

```
def get_neighbors(node):
```

```
    neighbors = []
```

```
    row, col = node
```

```
    for dr, dc in directions:
```

```
        nr = row + dr
```

```
        nc = col + dc
```

```
        if (
```

```
            0 <= nr < rows and
```

```
            0 <= nc < cols and
```

```
            maze[nr][nc] != 'X'
```

```
        ):
```

```
            neighbors.append((nr, nc))
```

```
    return neighbors
```

```
# -----
```

```
# UNIFORM COST SEARCH
```

```
# -----
```

```
def uniform_cost_search():
```

```

# Priority Queue:
# (cost, current node, path)

pq = [
    (0, start, [start])
]

best_cost = {
    start: 0
}

print("\n" + "=" * 45)
print("UNIFORM COST SEARCH")
print("=" * 45)

while pq:

    cost, current, path = heapq.heappop(pq)

    # Ignore outdated entries
    if cost > best_cost[current]:
        continue

    print(
        "Expanding:",
        current,
        "| Cost:",
        cost
    )

```

```

)

# Goal test
if current == goal:

    print("Goal Found by UCS!")

    return path, cost

for neighbor in get_neighbors(current):

    # Each movement costs 1
    new_cost = cost + 1

    if (
        neighbor not in best_cost or
        new_cost < best_cost[neighbor]
    ):

        best_cost[neighbor] = new_cost

    heapq.heappush(
        pq,
        (
            new_cost,
            neighbor,
            path + [neighbor]
        )
    )

```

```
return None, float('inf')
```

```
# -----
```

```
# DEPTH-LIMITED SEARCH FOR IDS
```

```
# -----
```

```
def depth_limited_search(
```

```
    current,
```

```
    goal,
```

```
    limit,
```

```
    path
```

```
):
```

```
    if current == goal:
```

```
        return path
```

```
    if limit == 0:
```

```
        return None
```

```
    for neighbor in get_neighbors(current):
```

```
        # Avoid cycles in current path
```

```
        if neighbor not in path:
```

```
            result = depth_limited_search(
```

```
                neighbor,
```

```
                goal,
```

```
        limit - 1,  
        path + [neighbor]  
    )
```

```
    if result is not None:  
        return result
```

```
    return None
```

```
# -----  
# ITERATIVE DEEPENING SEARCH  
# -----
```

```
def iterative_deepening_search():
```

```
    print("\n" + "=" * 45)  
    print("ITERATIVE DEEPENING SEARCH")  
    print("=" * 45)
```

```
    # Maximum useful simple-path depth  
    max_depth = rows * cols
```

```
    for depth in range(max_depth + 1):
```

```
        print(  
            "Searching with Depth Limit:",  
            depth  
        )
```

```
result = depth_limited_search(  
    start,  
    goal,  
    depth,  
    [start]  
)
```

```
if result is not None:
```

```
    print(  
        "Goal Found at Depth:",  
        depth  
    )
```

```
    return result
```

```
return None
```

```
# -----
```

```
# DISPLAY MAZE
```

```
# -----
```

```
print("=" * 45)
```

```
print("UNKNOWN MAZE")
```

```
print("=" * 45)
```

```
for row in maze:
```

```
print(" ".join(row))
```

```
print("\nLegend:")
```

```
print("S = Start")
```

```
print("G = Exit")
```

```
print("X = Obstacle")
```

```
print(". = Free Cell")
```

```
# -----
```

```
# RUN UCS
```

```
# -----
```

```
ucs_path, ucs_cost = uniform_cost_search()
```

```
# -----
```

```
# RUN IDS
```

```
# -----
```

```
ids_path = iterative_deepening_search()
```

```
# -----
```

```
# FINAL RESULTS
```

```
# -----
```

```
print("\n" + "=" * 45)
```

```
print("FINAL RESULTS")
```

```
print("=" * 45)
```

```
if ucs_path:
```

```
    print("\nUCS Path:")
```

```
    for i in range(len(ucs_path)):
```

```
        print(ucs_path[i], end="")
```

```
        if i < len(ucs_path) - 1:
```

```
            print(" -> ", end="")
```

```
    print("\nUCS Total Cost:", ucs_cost)
```

```
else:
```

```
    print("\nUCS: No Path Found")
```

```
if ids_path:
```

```
    print("\nIDS Path:")
```

```
    for i in range(len(ids_path)):
```

```
        print(ids_path[i], end="")
```

```
        if i < len(ids_path) - 1:
```

```
            print(" -> ", end="")
```

```
print(  
    "\nIDS Path Depth:",  
    len(ids_path) - 1  
)
```

else:

```
print("\nIDS: No Path Found")
```

4. OUTPUT

When the program is executed, an output similar to the following is obtained:

```
=====
```

UNKNOWN MAZE

```
=====
```

S . X . .

. . X . .

X . . . X

. X X . .

. . . . G

Legend:

S = Start

G = Exit

X = Obstacle

. = Free Cell

```
=====
```

UNIFORM COST SEARCH

```
=====
```

Expanding: (0, 0) | Cost: 0
Expanding: (0, 1) | Cost: 1
Expanding: (1, 0) | Cost: 1
Expanding: (1, 1) | Cost: 2
Expanding: (2, 1) | Cost: 3
Expanding: (2, 2) | Cost: 4
Expanding: (2, 3) | Cost: 5
Expanding: (1, 3) | Cost: 6
Expanding: (3, 3) | Cost: 6
Expanding: (0, 3) | Cost: 7
Expanding: (1, 4) | Cost: 7
Expanding: (3, 4) | Cost: 7
Expanding: (4, 3) | Cost: 7
Expanding: (0, 4) | Cost: 8
Expanding: (4, 2) | Cost: 8
Expanding: (4, 4) | Cost: 8

Goal Found by UCS!

ITERATIVE DEEPENING SEARCH

Searching with Depth Limit: 0
Searching with Depth Limit: 1
Searching with Depth Limit: 2
Searching with Depth Limit: 3
Searching with Depth Limit: 4

Searching with Depth Limit: 5

Searching with Depth Limit: 6

Searching with Depth Limit: 7

Searching with Depth Limit: 8

Goal Found at Depth: 8

FINAL RESULTS

UCS Path:

(0, 0) -> (0, 1) -> (1, 1) ->

(2, 1) -> (2, 2) -> (2, 3) ->

(3, 3) -> (3, 4) -> (4, 4)

UCS Total Cost: 8

IDS Path:

(0, 0) -> (0, 1) -> (1, 1) ->

(2, 1) -> (2, 2) -> (2, 3) ->

(3, 3) -> (3, 4) -> (4, 4)

IDS Path Depth: 8

5. OUTPUT ANALYSIS

Both search algorithms successfully find a path from:

Start = (0,0)

to:

Goal = (4,4)

without entering any obstacle cell.

The UCS path is:

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,2) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (4,4)$

The total number of movements is:

8

Since every movement costs:

1

the total UCS path cost is:

$$8 \times 1 = 8$$

Therefore:

UCS Total Cost = 8

IDS gradually increases its search limit from:

$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow 8$

At depth 8, it successfully finds the exit.

Therefore:

IDS Solution Depth = 8

The program demonstrates that both algorithms can successfully solve the given unit-cost maze.

UCS is particularly suitable when the movement costs may vary because it selects nodes according to accumulated cost.

IDS has an important memory advantage because its space complexity is approximately:

$O(bd)$

rather than storing a large frontier.

6. CONCLUSION

The unknown maze navigation problem was successfully solved using Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).

UCS expands nodes according to their accumulated path cost and finds a minimum-cost path when movement costs are positive. In the demonstrated maze, every movement costs 1, and UCS finds the exit with a total cost of 8.

IDS repeatedly performs depth-limited searches with increasing limits. It finds the exit at a depth of 8 while requiring significantly less memory than search strategies that store a large frontier.

For an intelligent robot operating in an unknown maze, a Model-Based Goal-Oriented Agent is an effective design. The model-based component can maintain information about explored paths, walls, and dead ends, while the goal-based component keeps the robot focused on reaching the exit.

If different paths have different movement costs, UCS is the preferred search strategy because it can find the minimum-cost route. If all movements have equal cost and the robot has strict memory limitations, IDS is a suitable alternative.

Therefore, combining a Model-Based Goal-Oriented Intelligent Agent with UCS provides an effective general solution for intelligent maze navigation, while IDS remains useful when memory efficiency is the primary requirement.

```
1 from collections import deque
2 cell
3
4 grid = [
5     ['S', '.', '.', 'X', '.', '.'],
6     ['X', '.', 'X', '.', 'X', '.'],
7     ['.', '.', 'X', '.', 'X', '.'],
8     ['.', 'X', 'X', '.', 'G'],
9     ['.', '.', '.', '.', '.']
10 ]
11
12 rows = 5
13 cols = 5
14
15 start = (0, 0)
16 goal = (4, 4)
17
18 # Movement: Up, Down, Left, Right
19 directions = [
20     (-1, 0), # Up
21     (1, 0), # Down
22     (0, -1), # Left
23     (0, 1) # Right
24 ]
25
26 # Manhattan Distance
27 def manhattan(node, goal):
28     return abs(node[0] - goal[0]) + abs(node[1] - goal[1])
29
30
31 # BFS Algorithm
32 def bfs(start, goal):
33     queue = deque([start])
34     visited = {start}
35     parent = {start: None}
36
37     step = 1
38
39     print("ROBOT PATH FINDING USING BFS")
40     print("=" * 40)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.848 Mb

Your Output

```
5 x 5 GRID
=====
S . . .
X . X .
. . . X
. X X .
. . . G
ROBOT PATH FINDING USING BFS
=====
```